

Computer Hardware

Threads and Synchronisation in C++

Ksawery Moźdzynski

Agenda For Today

1. Threads

Several threads, one program, many cores

2. The race

Why `counter++` loses updates

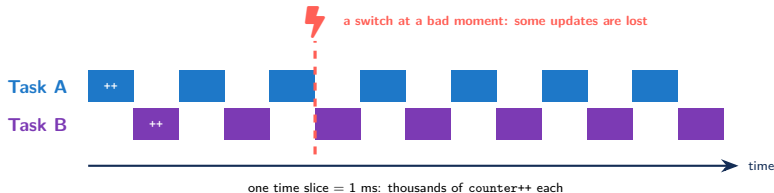
3. Mutexes and atomics

Three ways to fix it, and what they cost

4. The bank

A bigger program, and a new problem

Where we stopped: the puzzle



Simulator 6

Two tasks, six additions. The counter showed **4 or 5**.

Task 5

Two tasks, two million additions. The counter got **less**, and a different number every time.

The question

`counter++` is **one line**. How can a switch fall *in the middle* of it? Today: the answer, and the fix.

Task 0: C++ on your laptop

Task

Compile and run `hello.cpp` from Monday's last slide.

```
● ● ●
#include <iostream>
#include <thread>

int main() {
    std::thread t([] {
        std::cout << "Hello from a thread\n";
    });
    t.join();
}
```

g++ or clang++ (Linux, macOS, MinGW)

```
● ● ●
g++ -std=c++20 -O0 -pthread hello.cpp -o hello
./hello
```

MSVC (Developer Command Prompt)

```
● ● ●
cl /std:c++20 /EHsc /Od hello.cpp
hello.exe
```

From today we write C++ on our own laptops: **many cores, real threads.**
Does it not compile? **Raise your hand now**, not in an hour.

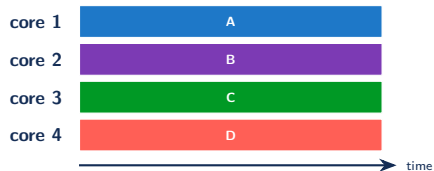
One core takes turns, many cores work at once

ESP32-C3: 1 core



Concurrency: tasks make progress by **taking turns**. This is what we saw on Monday.

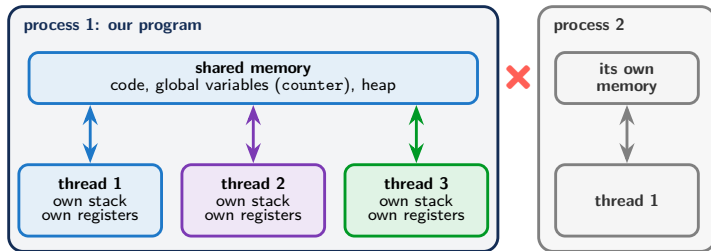
laptop: 4 cores



Parallelism: tasks run **at the same moment** on different cores.

A laptop does both: the cores work in parallel, and when there are more tasks than cores, they take turns.

Processes and threads



Process

A running program with **its own memory**. Other processes cannot see it.

Thread

A path of execution **inside** a process. The threads of one process **share** its memory, but each has its own stack and registers.

A thread is the laptop's version of Monday's **task**: its own context, shared global variables.

Task 1: how many cores?

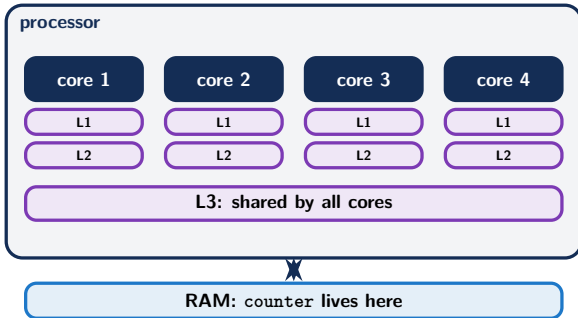
```
● ● ●  
#include <iostream>  
#include <thread>  
  
int main() {  
    std::cout << std::thread::hardware_concurrency()  
               << "\n";  
}
```

Task

Compile and run the program.

1. How many threads can your laptop run **at the same moment**?
2. Check how many **cores** your processor has: Task Manager (Windows), `sysctl -n hw.physicalcpu` (macOS), `lscpu` (Linux). Is it the same number?

Solution: how many cores?



Question 1

The number of **logical** cores: often 4 to 16.

Question 2

Often **twice** the number of cores: with *hyper-threading* (Intel, AMD) one core runs 2 threads. Apple M chips have no hyper-threading: the numbers are the same.

More threads than this number? They **take turns**: Monday's scheduler, now inside the operating system.

Creating a thread: `std::thread`

```
● ● ●
#include <iostream>
#include <thread>

void hello(int id) {
    std::cout << "Hello from thread " << id << "\n";
}

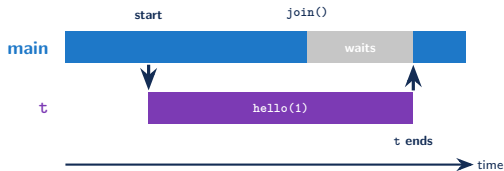
int main() {
    std::thread t(hello, 1);    // starts now
    std::cout << "Hello from main\n";
    t.join();                  // wait until t ends
}
```

`std::thread t(hello, 1)`

A new thread starts at once and runs `hello(1)`. `main` goes on at the same time.

`t.join()`

Wait until the thread ends. **Every** thread must be joined.



Like `xTaskCreate` on Monday: a function and a parameter. The operating system schedules it.

Many threads and their arguments

Four threads



```
#include <vector>

std::vector<std::thread> threads;
for (int i = 0; i < 4; i++)
    threads.emplace_back(hello, i); // start
for (auto &t : threads)
    t.join();                       // wait
```

A reference: std::ref



```
void addTo(int &total, int x) { total += x; }

int total = 0;
std::thread t(addTo, std::ref(total), 5);
```

`emplace_back(hello, i)`

Creates a new thread at the end of the vector. It starts at once.

Arguments are copied

Each thread gets its **own copy** of `i`.

`std::ref(total)`

Pass a **reference**: the thread changes the variable of `main`.

You will also see `[] { ... }`, as in `hello.cpp`: a *lambda*, a function without a name.

Task 2: four threads say hello

Task

Write a program that starts **4 threads**. Thread `i` prints `Hello` from thread `i`.

1. Run it **5 times**. Is the order of the lines always the same?
2. Do you ever see two messages **mixed** in one line?
3. Remove the loop with `join()`. What happens?

If you finish early: pass `total` to all 4 threads with `std::ref`. Thread `i` adds `i` to it. Print `total` after the joins. Do you always get 6?

Solution: four threads say hello



```
Hello from thread 1
Hello from thread 0
Hello from thread 3
Hello from thread 2
```



```
Hello from thread Hello from thread 2
0
```

Question 1

No. The operating system decides when each thread runs.

Question 2

Sometimes yes: << prints in **pieces**, and another thread can print between them.

Question 3

It crashes: `terminate` called without an active exception. Every thread must be joined.

If you finished early

Usually 6, but **not guaranteed**: see the next task.

With threads the order of events is **not fixed**. A program must be correct in **every** order.

Task 3: Monday's counter in C++

race.cpp

```
● ● ●  
#include <iostream>  
#include <thread>  
  
int counter = 0;  
const int N = 1'000'000;  
  
void work() {  
    for (int i = 0; i < N; i++)  
        counter++;  
}  
  
int main() {  
    std::thread a(work), b(work);  
    a.join(); b.join();  
    std::cout << "Goal: " << 2 * N << "\n";  
    std::cout << "Got:  " << counter << "\n";  
}
```

Task

Download `race.cpp` from the course page. Compile it with `-O0`.

1. Run it **5 times**. Write down the results.
2. Start only **one** thread and print `N` as the goal. Is the result correct now?

If you finish early: compile with `-O2` instead. What happens to the result, and to the time?

Solution: Monday's counter in C++

Two threads, 5 runs (a test machine)

Run	Got
1	1 322 869
2	1 484 050
3	1 334 267
4	2 000 000
5	1 863 852

Question 1

Usually **too small**, and different every time. Sometimes even correct: the first thread ended before the second one started.

Question 2

One thread: **always** correct.

If you finished early

With `-O2` the compiler adds `N` at once, without the loop. The result is (almost always) correct, but the bug is still there.

On Monday one core took turns. Today two cores work **at the same moment**. Same bug. Now: **why?**

counter++ is three instructions

C++

● ● ●
counter++;



RISC-V, as in Ripes

● ● ●
`lw` t0, 0(a0) *# 1. read counter from memory*
`addi` t0, t0, 1 *# 2. add 1 in a register*
`sw` t0, 0(a0) *# 3. write counter back*

Read, add, write

The processor cannot add to a value **in memory**. It reads it into a register, adds there, and writes it back. x86 and ARM do the same.

Between the instructions

The other thread can read or write counter: on one core after a **switch**, on two cores **at the same moment**.

One line of C++, but **three** steps for the processor. Things can happen **between** them.

One update lost, step by step

Step	Thread A	Thread B	counter
1	lw: t0 = 5		5
2		lw: t0 = 5	5
3	addi: t0 = 6		5
4		addi: t0 = 6	5
5	sw: writes 6		6
6		sw: writes 6	6

⚠ Two additions, but +1

Each thread worked on **its own copy** in its own register t0. B wrote over the result of A.

Why so many are lost

If a thread is stopped between lw and sw, the other one may add **thousands** of times. Then the first writes its old value over all of them.

This is the answer to Monday's puzzle.

Simulator 8 and Compiler Explorer

Simulator task

Topic **Race step by step**: threads A and B each run `lw, addi, sw` once. `counter = 5`. You choose which thread runs the next instruction.

1. Find an order that gives **7**. Then one that gives **6**.
2. Can you get **5**? Why?
3. How many of all possible orders give the correct result?

Then

Open godbolt.org. Paste the code below. Compiler: **RISC-V gcc**, option `-O1`.

```
int counter;  
void inc() { counter++; }
```

4. Find the three instructions.
5. Change the compiler to **x86-64 gcc**. How many instructions now? Is it safe now?

Solution: Simulator 8 and Compiler Explorer

Question 1

7: one thread does all three instructions first (A A A B B B). 6: both read before the other one writes (A B A B A B, ...).

Question 2

No. Each thread adds 1 to a value it has read, and that value is at least 5.

Question 3

There are 20 orders. Only 2 are correct: A A A B B B and B B B A A A.

Question 4

lw, addi, sw. The lui before them only builds the address of counter.

Question 5

One instruction: add DWORD PTR counter[rip], 1. Still **not safe**: inside the core it is still read, add, write, and another core can come in between.

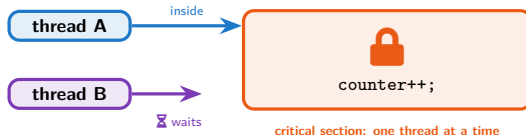
18 of 20 orders are wrong. That is why the error appears **so often**.

Race condition, data race, critical section

Race condition: the result depends on the **timing** of the threads.

Data race: two threads use the same variable at the same time, and at least one of them **writes**. In C++ this is **undefined behaviour**: anything may happen.

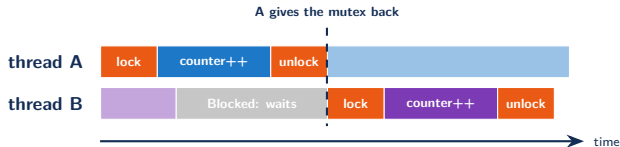
Critical section: a piece of code that must not run in two threads at the same time. Here: `counter++`.



The fix: make sure that only **one thread at a time** is inside the critical section.

A mutex: one thread at a time

A **mutex** (*mutual exclusion*) is a lock. `lock()`: wait until it is free, then take it. `unlock()`: give it back.



Only one owner

While A holds the mutex, B cannot take it. B can enter only after A's `unlock()`.

Waiting costs nothing

A thread that waits for a mutex is **Blocked**, like a task on Monday: it uses no CPU time.

Simulator 9: a mutex

Simulator task

Topic **Mutex**: the same two threads, but now each runs `lock`, `lw`, `addi`, `sw`, `unlock`. `counter = 5`.

1. Try to find an order that gives **6**.
2. Thread A holds the mutex. Click thread B. What happens? In which state is B?
3. How many instructions can come **between** A's `lw` and A's `sw` now?

Solution: Simulator 9

Question 1

Impossible: **every** order gives 7.

Question 2

B stops at lock. It is **Blocked** until A calls `unlock`.

Question 3

None of thread B. The three steps of A now act like **one** step.

The mutex removes all the wrong orders. Now let us write it in C++.

std::mutex and std::lock_guard

```
● ● ●
#include <mutex>

std::mutex m;

void work() {
    for (int i = 0; i < N; i++) {
        std::lock_guard<std::mutex> lock(m);
        counter++;
    } // lock is destroyed here: unlock
}
```

std::lock_guard

Locks the mutex when it is created, and unlocks it at the end of the block { ... }.

⚠ Why not m.lock() and m.unlock()?

It is easy to forget unlock(), for example after an early return. Then every other thread waits **forever**.

One mutex protects **one** piece of shared data. Every thread that uses the data must lock **the same** mutex.

Task 4: fix it with a mutex, and measure

Measure the time in main()

```
● ● ●  
#include <chrono>  
  
auto start = std::chrono::steady_clock::now();  
// ... start and join the threads ...  
auto end = std::chrono::steady_clock::now();  
std::chrono::duration<double, std::milli> ms  
    = end - start;  
std::cout << ms.count() << " ms\n";
```

⚠ **Not** `clock()` from Lecture 4
It adds up the CPU time of **all** threads.

Task

Start from Task 3 (-00).

1. Add a mutex. Is the result correct now?
2. Measure the time **without** and **with** the mutex.
3. How many times slower is the correct version?

If you finish early: put the `lock_guard` **before** the loop, so that the whole loop is locked. Correct? How fast? What do the threads do now?

Solution: fix it with a mutex

Two threads (a test machine)

Version	Got	Time
no protection	1 334 267	6 ms
mutex	2 000 000	200 ms

Question 1

Yes: exactly 2 000 000, every time.

Questions 2 and 3

About **30 times** slower. Every `counter++` now needs `lock` and `unlock`, and the threads wait for each other all the time.

If you finished early

Correct and fast, but the threads run **one after the other**: no parallelism at all.

Correct, but slow. Can we make the three steps **one** step instead?

Atomic operations

C++

```
● ● ●  
#include <atomic>  
  
std::atomic<int> counter{0};  
  
void work() {  
    for (int i = 0; i < N; i++)  
        counter++;    // one step  
}
```

Atomic: the read, the add and the write happen as **one** step. No other thread can come in between.

RISC-V

```
● ● ●  
# before: three instructions  
lw    t0, 0(a0)  
addi  t0, t0, 1  
sw    t0, 0(a0)  
  
# now: one atomic instruction  
li     t0, 1  
amoadd.w zero, t0, (a0)    # counter += 1
```

The hardware helps

RISC-V: `amoadd.w` and others; and `lr.w / sc.w`: if someone wrote in between, try again.

An atomic works for **one** variable and simple operations. For more than that, we need a mutex.

Task 5: atomic, and a better idea

Task

Two more versions of the counter. Measure the time of each.

1. **Atomic:** `std::atomic<int>` counter, without the mutex.
2. **Local sum:** each thread counts in its **own local variable**, and adds it to the shared counter **once**, at the end (with the mutex).
3. Fill in the table. Which versions are correct? Which is the fastest correct one?

Version	Correct?	Time
no protection		
mutex		
atomic		
local sum		

If you finish early: in Compiler Explorer, make counter a `std::atomic<int>`. Which instruction do you see for RISC-V? And for x86-64?

Solution: atomic, and a better idea

Two threads (a test machine)

Version	Correct?	Time
no protection	✗	6 ms
mutex	✓	200 ms
atomic	✓	30 ms
local sum	✓	2.5 ms

If you finished early

RISC-V: amoadd.w.aqrl. x86-64: lock add. (aqrl and lock: ordering rules, not needed today.)

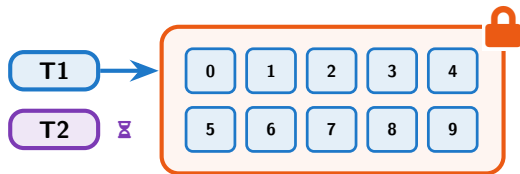
Local sum

```
void work() {  
    int local = 0;  
    for (int i = 0; i < N; i++)  
        local++; // nothing shared  
    std::lock_guard<std::mutex> lock(m);  
    counter += local; // once per thread  
}
```

The best synchronisation is the one you do not need: **share less**.

Lock granularity: one big lock, or many small ones?

One big lock

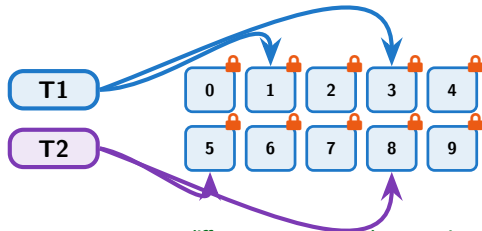


one transfer at a time

Simple and safe

But threads wait for each other, even when they use different data.

A lock for each piece of data



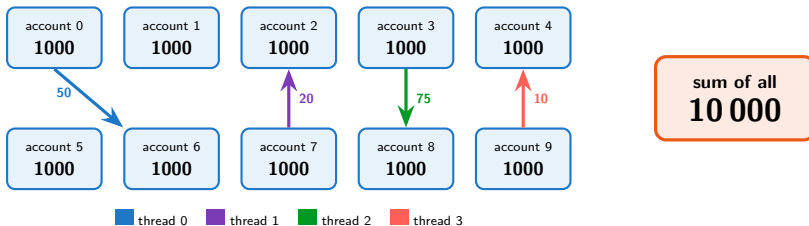
different accounts: at the same time

More work in parallel

Threads that use different data do not wait. But it is harder to get right.

Next: we try both in a bigger program.

The bank



A transfer

Pick two accounts and an amount. Subtract it from one account, add it to the other. Several threads do this at the same time.

The **sum** of all balances must always be **10 000**. Easy to check: if the sum is wrong, the program is wrong.

Negative balances are allowed: we keep the bank simple.

Bank, step 1: no synchronisation



```
const int ACCOUNTS = 10, THREADS = 4;
const int TRANSFERS = 1'000'000 / THREADS;
int balance[ACCOUNTS];

void transfers(int id) {
    std::mt19937 rng(id);
    std::uniform_int_distribution<int>
        acc(0, ACCOUNTS - 1), amount(1, 100);
    for (int i = 0; i < TRANSFERS; i++) {
        int from = acc(rng), to = acc(rng);
        if (from == to) continue;
        int x = amount(rng);
        balance[from] -= x;
        balance[to] += x;
        if (i % 50'000 == 0)
            std::cout << id << ": " << i << "\n";
    }
}
```

bank.cpp: main()

Not shown here. Sets every balance to 1000, starts THREADS threads, joins them, and prints the **sum** and the **time**.

Task

Download bank.cpp. Compile it with -O0.

1. Run it several times. What is the sum?
2. Set THREADS = 1. And now?

Solution, step 1: money disappears

4 threads (a test machine)



Sum: 37637 time: 100 ms

Sum: -117645 time: 115 ms

1 thread



Sum: 10000 time: 110 ms

Question 1

The sum is wrong, and different every time. Money **disappears** or **appears**.

Question 2

With one thread the sum is always 10 000.

! Why?

`balance[from] -= x` is the same **read, change, write** as `counter++`.

Bank, step 2: one mutex for the whole bank



```
std::mutex bank;
```

```
// in transfers(), around the transfer:  
std::lock_guard<std::mutex> lock(bank);  
balance[from] -= x;  
balance[to]   += x;
```

Task

Add one `std::mutex` for the whole bank.

1. Is the sum correct now?
2. Measure the time with `THREADS = 1, 2 and 4`.
The total number of transfers stays the same.
Does it get faster with more threads?

Solution, step 2: correct, but a queue

One mutex (a test machine)

Threads	Sum	Time
1	10 000	150 ms
2	10 000	210 ms
4	10 000	200 ms

Question 1

Yes: always 10 000.

Question 2

No. More threads are **not** faster, even a bit slower: only one transfer at a time, and the others wait at the lock.

A transfer from 1 to 2 and a transfer from 5 to 8 do not need to wait for each other. Idea: **one mutex per account**.

Bank, step 3: one mutex per account



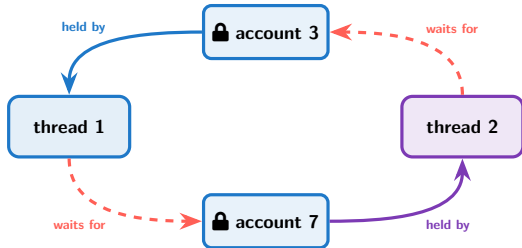
```
std::mutex m[ACCOUNTS];  
  
// in transfers(), around the transfer:  
std::lock_guard<std::mutex> a(m[from]);  
std::lock_guard<std::mutex> b(m[to]);  
balance[from] -= x;  
balance[to]   += x;
```

Task

Replace the bank mutex with a mutex for **each account**. A transfer locks its two accounts.

1. Run it several times. Does it always finish?
2. Watch the progress messages. What does the program do? Look at the CPU usage (Task Manager, Activity Monitor, top).

Solution, step 3: the program stops



Questions 1 and 2

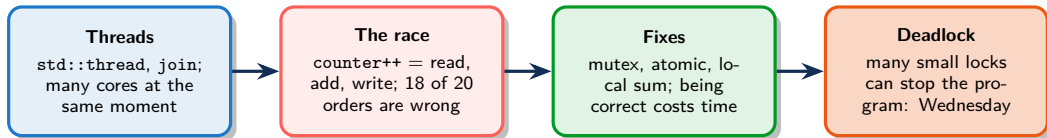
It **hangs**: the messages stop, the program never ends, and it uses **no** CPU time. The threads are not working, they are waiting.

Deadlock: each thread waits for a lock that another thread holds. Nobody can continue.

The question

How can a thread lock **two** accounts without this happening? **Tomorrow.**

What we saw today



Monday's puzzle is solved: `counter++` is not one step, and a lock makes it one.

Back on the ESP32: a FreeRTOS mutex



```
SemaphoreHandle_t m;
```

```
void setup() {  
    m = xSemaphoreCreateMutex();  
    // the LCD and the two tasks as before  
}
```

```
void incTask(void *param) {  
    for (int i = 0; i < N; i++) {  
        xSemaphoreTake(m, portMAX_DELAY);    // lock  
        counter++;  
        xSemaphoreGive(m);                    // unlock  
    }  
    ...  
}
```

Monday's Task 5, fixed

The LCD now shows Got: 2000000.

portMAX_DELAY

Wait as long as needed. The task is **Blocked** while it waits.

If you finish early: try it in your project **Race**.

The same idea everywhere: FreeRTOS, C++, every operating system.

Check yourself

1. What is the difference between a process and a thread? What do the threads of one process share?
2. Why does `counter++` lose updates? Use the words *read*, *add* and *write*.
3. Why is the mutex version correct but slow? What did the local-sum version do better?
4. What is a deadlock? Draw the situation from step 3 of the bank.

Keep your `bank.cpp`: on Wednesday we repair step 3.